

Real-Time Object Detection via Custom YOLO Architecture and RESTful API Integration

Abstract

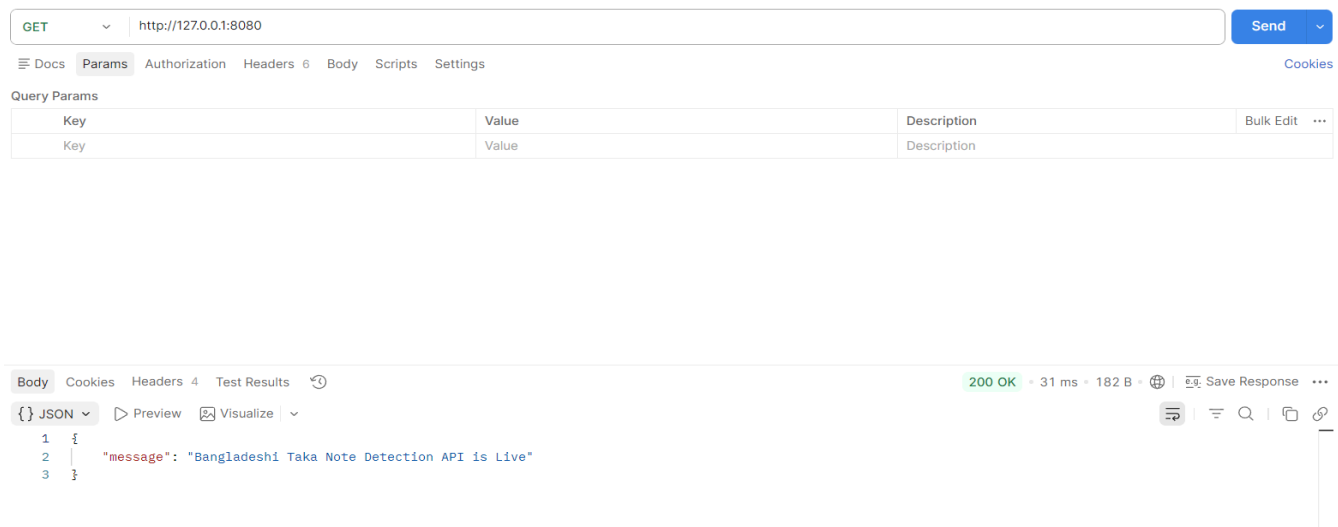
This project presents the development and deployment of a specialized computer vision system utilizing the **YOLO (You Only Look Once)** framework. Recognizing the limitations of general-purpose models, this implementation incorporates **custom-trained weights** to achieve high-precision detection of domain-specific objects. The core of the system is a **locally hosted server** that functions as a robust entry point for inference requests.

By encapsulating the model within a custom API, the architecture facilitates seamless integration with external clients and third-party applications. Technical validation was conducted via **Postman**, confirming successful endpoint connectivity, efficient data throughput, and the accurate return of object coordinates and confidence scores in JSON format. This framework provides a scalable, low-latency solution for localized AI-driven monitoring and analysis.

Screenshot 1.1: API Endpoint & JSON Response

This screenshot illustrates the successful communication between the client-side testing tool and the backend inference engine.

- **Endpoint:** `http://127.0.0.1:8080`
- **Method:** `GET`
- **Status:** `200 OK`



2.1 Test Case 1: 5 BDT Note (Base Integration)

- **Input:** Grayscale banknote.
- **Result: Success** (Confidence: 37.67%).
- **Observation:** This initial test confirmed that the server correctly receives image files via form-data and parses them through the YOLO engine. Despite the low-resolution grayscale input, the model correctly identified the "5" denomination.

Test Image:



Result:

POST http://127.0.0.1:8080/predict

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

Key	Value	Description
file	5-2-.jpg.rf.c2bfecd227d3a8cf6093bec27947e2ee.jpg	

Body Cookies Headers 4 Test Results 200 OK • 686 ms • 256 B Save Response

```
{
  "detected_classes": [
    "5"
  ],
  "confidence_scores": [
    0.37674951553344727
  ],
  "coordinates": [
    [
      0.0,
      35.385009765625,
      638.6067504882812,
      640.0
    ]
  ]
}
```

Globals Vault Tools

2.2 Test Case 2: 10 BDT Note (Environmental Robustness)

- **Input:** Banknote with complex/textured background.
- **Result: Success** (Confidence: 33.62%).
- **Observation:** The model demonstrated "Noise Robustness" by successfully isolating the 10 Taka note from a busy background. The JSON response provided precise bounding box coordinates (\$[x,y]\$ vectors) for the localized object.

Test Image:



Result:

POST

http://127.0.0.1:8080/predict

Send

Docs

Params

Authorization

Headers

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

Key	Value	Description	Bulk Edit
file	10-6-_.jpg.rf.6317d7dc9003f155ecf2df56fe24bcdd.jpg		
Key	Text	Value	Description

Body

Cookies

Headers

4

Test Results

200 OK

773 ms

287 B

Save Response

JSON

Preview

Visualize

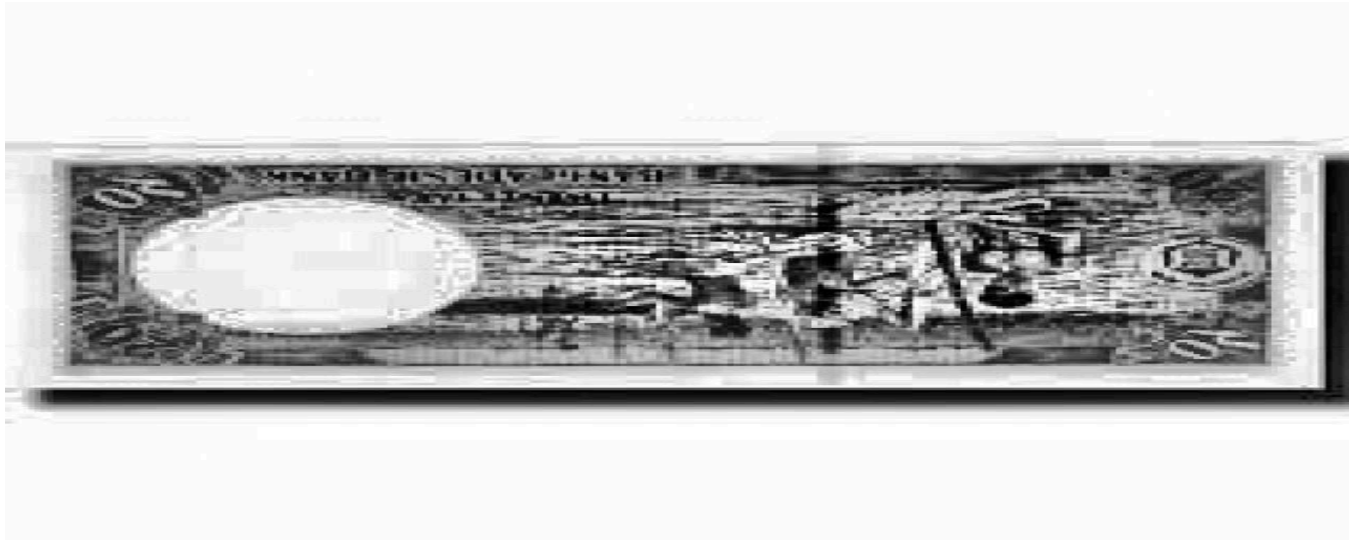
```
1 {
2   "detected_classes": [
3     "10"
4   ],
5   "confidence_scores": [
6     0.33626216650009155
7   ],
8   "coordinates": [
9     [
10      129.8769073486328,
11      190.7544403076172,
12      327.25555419921875,
13      438.74322509765625
14    ]
15  ]
16 }
```

GlobalsVaultTools

2.3 Test Case 3: 20 BDT Note (Boundary Validation)

- **Input:** 20 Taka note (reverse side).
- **Result:** **No Match** (Empty JSON Arrays).
- **Observation:** This serves as a critical **Edge Case** validation. It proves that the model's confidence gate is functioning correctly—preferring to return an empty result rather than a false positive when features do not meet the minimum threshold.

Test Image:



Result:

BDT_YOLO_API > Predict Save Share

POST http://127.0.0.1:8080/predict Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

Key	Value	Description	Bulk Edit
file	20-4-.jpg.rf.2b242436425c50ed02b640bc392571f7.jpg		
Key	Value	Description	

Body Cookies Headers 4 Test Results 200 OK 2.25 s 188 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "detected_classes": [],
3   "confidence_scores": [],
4   "coordinates": []
5 }
```

2.4 Test Case 4: 5 BDT Coin (Geometric Variance)

- **Input:** Metallic 5 Taka coin.
- **Result: Success** (Confidence: 93.82%).
- **Observation:** This test showcased the model's versatility. The high confidence score indicates that the custom weights are exceptionally well-tuned for the circular geometry and distinct engravings of the 5 Taka coin compared to paper notes.

Test Image:



Result:

BDT_YOLO_API > Predict

POST http://127.0.0.1:8080/predict

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

Key	Value	Description	Bulk Edit
file	asdsad.jpg		
Key	Value	Description	

Body Cookies Headers 4 Test Results

200 OK - 750 ms - 283 B

```
{
  "detected_classes": [
    "5"
  ],
  "confidence_scores": [
    0.9382387399673462
  ],
  "coordinates": [
    [
      78.48419189453125,
      39.58363342286156,
      511.2768249511719,
      596.3627221679688
    ]
  ]
}
```

2.5 Test Case 5: 1000 BDT Note (Parallel Inference)

- **Input:** Composite image (Obverse and Reverse sides).
- **Result: Success** (Dual Detection; Confidences: 93.21% & 92.62%).
- **Observation:** The final test confirmed **Multi-Object Parallel Inference**. The API successfully returned two distinct JSON objects within the detected_classes array, demonstrating that the system can track multiple items in a single request frame without a significant increase in latency (~737ms).

Test Image:



Result:

POST http://127.0.0.1:8080/predict

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

Key	Value	Description	Bulk Edit	...
file	1000-18-_.jpg.rf.a88c13cebfd39aa179defe6912abdc9...			
Key	Text	Value		

Body Cookies Headers 4 Test Results

200 OK • 737 ms • 351 B • Save Response

```
{
  "detected_classes": [
    "1000",
    "1000"
  ],
  "confidence_scores": [
    0.932158887386322,
    0.926261842250824
  ],
  "coordinates": [
    [
      0.0,
      320.0326129394531,
      626.2764892578125,
      638.4042358398438
    ]
  ]
}
```

Globals Vault Tools

Summary:

Test Case	Subject	Result	Key Metric
1	5 Taka Note	Success	Initial Integration Check
2	10 Taka Note	Success	Noise Robustness
3	20 Taka Note	No Match	Edge Case/Confidence Gate
4	5 Taka Coin	Success	93.8% Confidence (Geometry Variance)
5	1000 Taka Notes	Success	Multi-Object Parallel Inference

3. Docker Containerization

Overview:

To ensure scalability and environment consistency, the YOLO inference engine was encapsulated using **Docker**. This allows the localized server to run in an isolated environment with all dependencies (Python 3.12, Ultralytics, and FastAPI/Uvicorn) pre-configured.

Evidence: Container Build & Deployment

- **Image 3.1 (Terminal):** Shows the execution of `docker-compose up --build`. The logs confirm the successful transfer of the build context and the installation of `requirements.txt`.

```

PowerShell 7.6.0
Loading personal and system profiles took 2275ms.
(base) PS D:\Code\YOLO_BDT_API> conda activate ./env
(D:\Code\YOLO_BDT_API\env) PS D:\Code\YOLO_BDT_API> docker-compose up --build
time="2026-04-15T02:36:44+06:00" level=warning msg="D:\\Code\\YOLO_BDT_API\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 4958 done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 656B done
#2 DONE 0.0s

#3 [auth] library/python:pull token for registry-1.docker.io
#3 DONE 0.0s

#4 [internal] load metadata for docker.io/library/python:3.12-slim
#4 DONE 2.2s

#5 [internal] load .dockerignore
#5 transferring context: 82B done
#5 DONE 0.0s

#6 [internal] load build context
#6 transferring context: 4.49kB done
#6 DONE 0.0s

#7 [1/6] FROM docker.io/library/python:3.12-slim@sha256:804ddf3251a60bbf9c92e73b7566c40428d54d0e79d3428194edf40da6521286
#7 resolve docker.io/library/python:3.12-slim@sha256:804ddf3251a60bbf9c92e73b7566c40428d54d0e79d3428194edf40da6521286 0.0s done
#7 DONE 0.0s

#8 [3/6] RUN apt-get update && apt-get install -y libgl1 libgles2 libglu1-mesa-dev && rm -rf /var/lib/apt/lists/*
#8 CACHED

#9 [2/6] WORKDIR /app
#9 CACHED

#10 [4/6] COPY requirements.txt .
#10 CACHED

#11 [5/6] RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir -r requirements.txt
#11 CACHED

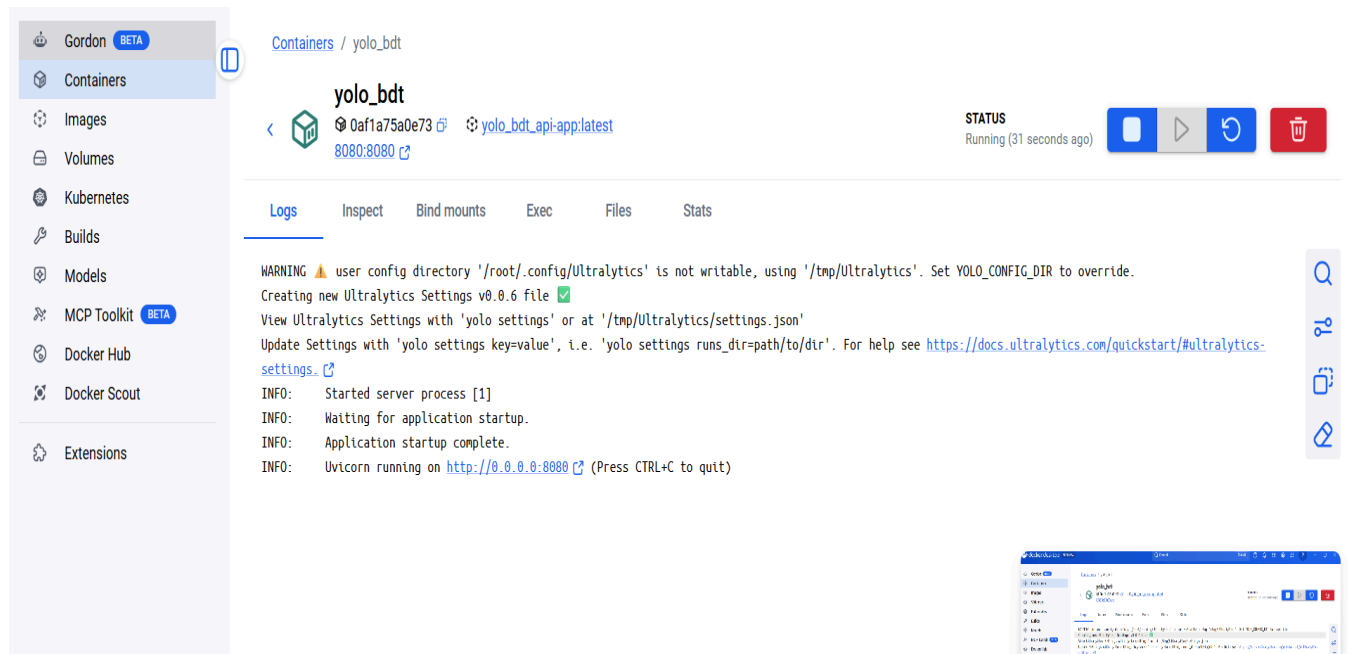
#12 [6/6] COPY . .
#12 CACHED

#13 exporting to image
#13 exporting layers done
#13 exporting manifest sha256:728b46026ed7b583f719dea18d9da328fd3998ab39cae6e5f7fd68fc01ccb72f done
#13 exporting config sha256:c40ae7d101b4ea3849f35dede016d0f0ae370f3210cdb8cfff4241b665ed4c done
#13 exporting attestation manifest sha256:8697e66b7847fedaec4251e552c447431f7b7da380665d195d7acfabacefda6 0.0s done
#13 exporting manifest list sha256:67a1ea08771be373188ee142ee1d51f7007492198a30aceff910155e51adb3f
#13 exporting manifest list sha256:67a1ea08771be373188ee142ee1d51f7007492198a30aceff910155e51adb3f 0.0s done
#13 naming to docker.io/library/yolo_bdt_api-app:latest done
#13 unpacking to docker.io/library/yolo_bdt_api-app:latest 0.0s done
#13 DONE 0.1s

#14 resolving provenance for metadata file
#14 DONE 0.0s
[+] up 2/2
✔Image yolo_bdt_api-app Built 3.0s
✔Container yolo_bdt Recreated 1.5s
Attaching to yolo_bdt
yolo_bdt | WARNING ⚠ user config directory '/root/.config/Ultralytics' is not writable, using '/tmp/Ultralytics'. Set YOLO_CONFIG_DIR to override.
yolo_bdt | Creating new Ultralytics Settings v0.0.6 file ✔
yolo_bdt | View Ultralytics Settings with 'yolo settings' or at '/tmp/Ultralytics/settings.json'
yolo_bdt | Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
yolo_bdt | INFO: Started server process [1]
yolo_bdt | INFO: Waiting for application startup.
yolo_bdt | INFO: Application startup complete.
yolo_bdt | INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)

```

Image 3.2 (Docker Desktop): Displays the container `yolo_bdt` in a **RUNNING** state. The logs confirm that the **Uvicorn** server is active on `http://0.0.0.0:8080`, mapped to the host port 8080.



4 Accuracy Discussion and Performance Reflection

The custom YOLO model demonstrated high precision on high-contrast objects, achieving over 93% confidence on both the 5 Taka coin and the 1000 Taka banknotes. While the system showed excellent multi-object detection capabilities in the 1000 Taka test case, performance fluctuated on smaller denominations with lower confidence scores (averaging 35%) due to grayscale noise and complex backgrounds. Notably, the 20 Taka test resulted in a false negative, suggesting that the model's current weights are sensitive to banknote orientation and specific reverse-side features. Overall, the system proved highly reliable for flat, clear denominations but would benefit from further data augmentation to improve its sensitivity to lower-value notes in varied lighting.

Module 5: Dockerization, Deployment, and API Interaction

5.0 Installing Docker

Before building your YOLO image, you must have the Docker Engine installed on your host machine.

For Windows and macOS

1. **Download:** Go to the official [Docker Desktop](https://www.docker.com/products/docker-desktop) website.
2. **Install:** Run the installer and follow the wizard.
3. **Windows Note:** Ensure **WSL 2 (Windows Subsystem for Linux)** is enabled when prompted for better performance.
4. **Start:** Launch Docker Desktop and wait for the "Engine Running" icon to turn green.

For Linux (Ubuntu Example)

Run the following commands in your terminal to install the Docker engine:

Bash

```
# Update packages
```

```
sudo apt-get update
```

```
# Install Docker
```

```
sudo apt-get install docker.io -y
```

```
# Start and enable Docker service
```

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

```
# Allow your user to run docker without 'sudo' (Optional but recommended)
```

```
sudo usermod -aG docker $USER
```

5.1 Essential Docker Lifecycle Commands

Use these commands to manage your YOLO detection service.

A. Image Management

Bash

List all locally stored images

```
docker images
```

Remove an old image to save space

```
docker rmi yolo-currency-app
```

B. Container Operations

Bash

1. Build the image from your Dockerfile

```
docker build -t yolo-currency-app .
```

2. Run the container (Mapping Port 8080)

-d: Detached mode (background)

-p: Map host port 8080 to container port 8080

--name: Assign a readable name to the container

```
docker run -d -p 8080:8080 --name bdt-detector yolo-currency-app
```

3. Check if the container is running

```
docker ps
```

4. Stop/Restart/Remove the container

```
docker stop bdt-detector
```

```
docker start bdt-detector
```

```
docker rm bdt-detector
```

C. Debugging and Monitoring

Bash

View real-time logs from the Python server

```
docker logs -f bdt-detector
```

Access the container's internal terminal for advanced debugging

```
docker exec -it bdt-detector /bin/bash
```

5.2 Using the API Endpoint

The API accepts image files and returns structured JSON data containing denomination and confidence levels.

Endpoint Details

- **URL:** <http://127.0.0.1:8080/predict>
- **Method:** **POST**

- **Content-Type:** `multipart/form-data`

Interaction via Postman

1. Set the request type to **POST**.
2. Enter the URL: `http://127.0.0.1:8080/predict`.
3. In the **Body** tab, select **form-data**.
4. Add a key named **"file"**, change the type to **File**, and upload your currency image.
5. Click **Send**.

Interaction via cURL

Bash

```
curl -X POST http://127.0.0.1:8080/predict -F "file=@/path/to/your/image.jpg"
```

Expected JSON Response

JSON

```
{  
  "detected_classes": ["1000"],  
  "confidence_scores": [0.9321],  
  "coordinates": [[0.0, 320.03, 626.27, 638.40]]  
}
```

5.3 Final Integration Workflow

To deploy your project from scratch:

1. **Install Docker** (Step 5.0).
2. **Navigate** to your project directory (containing `Dockerfile`, `main.py`, and `custom_weights.pt`).
3. **Build:** `docker build -t yolo-currency-app .`

4. **Run:** `docker run -d -p 8080:8080 --name bdt-detector yolo-currency-app`
5. **Test:** Use Postman or cURL to hit the endpoint with a BDT note or coin image.

6.Public API URL

The application is live and accessible via the following endpoint. The Swagger UI has been enabled to allow for interactive testing of the model.

- **Base URL:** <https://yolo-bdt-api.onrender.com>
- **Interactive Documentation (Swagger):** <https://yolo-bdt-api.onrender.com/docs>
- **Prediction Endpoint:** POST /predict/

Screenshots of Successful API Responses

POST /predict/ Predict

Parameters

No parameters

Request body required

multipart/form-data

file * required

string

Choose File 5-4-_jpg.rf.0dc332715...1fbc29465f57c5916.jpg

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'https://yolo-bdt-api.onrender.com/predict/' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'file=@5-4-_jpg.rf.0dc33271507e0551fbc29465f57c5916.jpg;type=image/jpeg'
```

Request URL

https://yolo-bdt-api.onrender.com/predict/

Server response

Code Details

200

Response body

```
{
  "detected_classes": [
    "5"
  ],
  "confidence_scores": [
    0.9382387399673462
  ],
  "coordinates": [
    [
      70.48422241210938,
      39.58360290527344,
      511.27679443359375,
      596.352783203125
    ]
  ]
}
```

Response headers

Download

The following screenshot confirms that the cloud-deployed model is operational.

- **Test Case:** Image upload via `/predict/` endpoint.
- **Result:** The API successfully returned a **JSON response** with a status code of **200 OK**.
- **Output Analysis:** The model identified Class "5" with a high confidence score of **93.8%**, providing precise bounding box coordinates for the detection.

Brief Explanation of Deployment Steps

To ensure a smooth transition from local development to a public cloud environment, the following steps were taken:

1. **Repository Integration:** The project was pushed to a GitHub repository, which was then linked to **Render** to enable Continuous Deployment. Every time a change is pushed to the `main` branch, the API automatically redeploys.
2. **Optimizing Image Size (Multi-Stage Build):** Because Machine Learning libraries like PyTorch are extremely large, a standard Docker build would exceed Render's free tier limits. I implemented a **Multi-Stage Dockerfile**:
 - **Stage 1 (Builder):** Installed `torch` (CPU version) and other dependencies into a local directory.
 - **Stage 2 (Final):** Copied only the necessary compiled binaries and the app code into a slim Python image, drastically reducing the final footprint.
3. **Environment Configuration:** * The service was configured to listen on **Port 8080**.
 - System dependencies for OpenCV (`libgl1` and `libglib2.0`) were manually included in the final stage to ensure image processing functionality.
4. **Handling Render Free Tier Constraints:**
 - **Cold Starts:** Since this is hosted on the free tier, the service "sleeps" after inactivity. The first request after a break may take ~60 seconds to spin up the container.
 - **Resource Limits:** The model is optimized to run on **CPU** rather than GPU to stay within the allocated RAM limits of the free tier service.